Introduction to DevSecOps · Midterm test

Option A. Lessons 1–7.

Name: Group: Date:

Choose **one** answer for each question and write its letter in the table below. No laptop, no phone. Each correct answer gives 4 points, 40 points in total.

1	2	3	4	5	6	7	8	9	10

1. What does “idempotent” mean for a script or a playbook?
   - **A** Running it once or many times leaves the server in the same state
   - **B** It can run only once; the second run fails
   - **C** It runs on many servers at the same time
   - **D** It stops at the first command that fails

2. Why does the API run as the user `conduit`, not as `azureuser`?
   - **A** Ubuntu allows one program per user, and `azureuser` already runs the web server
   - **B** Only `conduit` may open port 80; `azureuser` may not
   - **C** An attacker inside the API gets only the very small rights of `conduit`
   - **D** Programs started by `azureuser` stop when the SSH session closes, so the API would go down

3. A stranger opens a pull request from their fork. Why is nothing released?
   - **A** The server accepts SSH only from the IP address of your laptop
   - **B** The tests always fail on a fork, so the deploy job never starts
   - **C** GitHub blocks every pull request from people outside your organization
   - **D** The deploy job runs only for a push to `main`, and pull requests from forks get no secrets

4. You rolled back by running the last green pipeline again. Why is a revert still needed?
   - **A** The rollback released only the API; the revert releases the frontend
   - **B** The rollback deleted the broken commit, and the revert brings the history back
   - **C** The broken commit is still the newest on `main`; the next merge would release it again
   - **D** GitHub refuses new merges until the broken commit is reverted

5. `sudo ss -tlnp` on a server shows the lines below. Which port can be reached **only by programs on the server itself**?

```
LISTEN  127.0.0.1:8000   uvicorn
LISTEN  *:3000           node
LISTEN  *:443            caddy
LISTEN  *:22             sshd
```

   - **A** 443
   - **B** 8000
   - **C** 3000
   - **D** 22

6. A Dockerfile has the line `ENV JWT_SECRET_KEY=super-secret-123`. What is wrong?
   - **A** The secret is baked into the image; anyone who can pull the image can read it
   - **B** Names in `ENV` cannot contain the underscore character
   - **C** Nothing: `ENV` values disappear when the build finishes
   - **D** Nothing: values set with `ENV` are encrypted inside the image

7. What is the difference between an image and a container?
   - **A** An image runs on Linux; a container runs on Windows and macOS
   - **B** A container is a small virtual machine with its own kernel; an image is its disk
   - **C** An image is the packed, frozen box; a container is that box running
   - **D** An image is for the frontend; a container is for the API

8. A real key was pushed to a public repository. What do you do first?
   - **A** Delete the commit and force-push the branch
   - **B** Make the repository private for a while
   - **C** Add the file with the key to `.gitignore`
   - **D** Revoke the key where it was issued and create a new one

9. `git grep` finds nothing, but gitleaks finds the key. Why?
   - **A** Gitleaks also searches other people's forks of the repository
   - **B** Gitleaks searches every commit in the history, and an old commit still has the key
   - **C** `git grep` skips files whose names start with a dot, like `.env`
   - **D** `git grep` searches only `main`, not your branch

10. Where on the path of a change (idea → code → tests → build → deploy → run → watch) does security belong?
    - **A** On every step of the path
    - **B** Only in the code, during code review
    - **C** Only at the end, just before the release
    - **D** Only on the server, at deploy and run